BM 算法坏字符规则

1. Boyer Moore 算法介绍

Boyer Moore 算法（BM 算法）：由 Robert S. Boyer 和 J Strother Moore 于 1977 年提出，是一种高效的字符串搜索算法，实际应用中通常比 KMP 算法快 3~5 倍.

该算法的核心思想是：先对模式串 p 预处理，生成辅助表。在匹配过程中，如果文本串 T 某字符与模式串 p 不匹配，通过移动规则，直接跳过不可能匹配的位置，将模式串向后滑动多位.

2. 坏字符规则

当文本串 T 和模式串 p 从右向左比较时，如果遇到第一个不匹配的字符（称为坏字符），可以利用该字符串快速决定模式串的滑动距离.

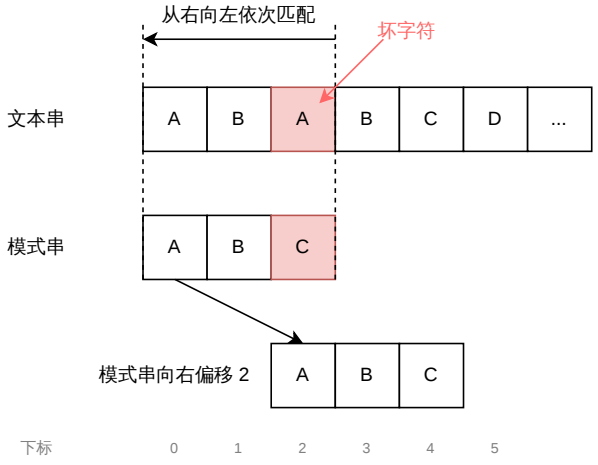

图1

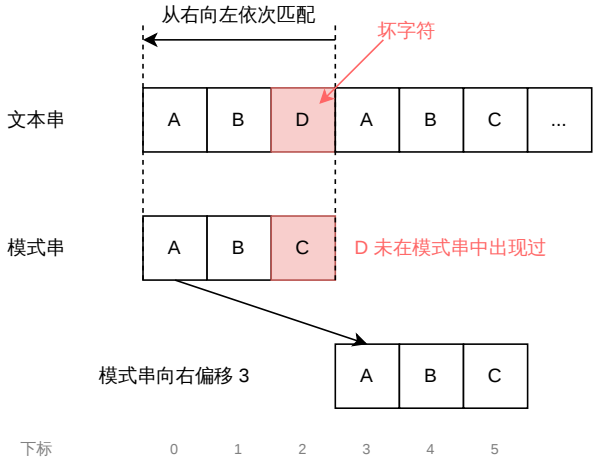

图2

情况 1：坏字符在模式串 p 中出现过.

- 将模式串最后一次出现该坏字符的位置与文本串中的坏字符对齐.
- 移动位数 $=$ 坏字符在模式串中的失配位置 $-$ 坏字符在模式串中最右侧出现的位置.

如图 1 所示：

1. 将模式串与文本串起始位置对齐，依次从右向左比较.
2. 发现坏字符 A，失配位置发生在模式串下标 2.
3. 查找坏字符在模式串中的最右位置，**即下标 0 处**.
4. 偏移量 $shift = 2 - 0 = 2$，模式串向右偏移 2.

情况 2：坏字符在模式串 p 中没有出现过.

- 直接将模式串向右移动.
- 移动位数 $=$ 坏字符在模式串中的失配位置 $+1$.

如图 2 所示：

1. 将模式串与文本串起始位置对齐，依次从右向左比较.
2. 发现坏字符 A，失配位置发生在模式串下标 2.
3. 查找坏字符在模式串中的最右位置，**坏字符不在模式串中**.
4. 偏移量 $shift = 2 + 1 = 3$，模式串向右偏移 3.

特殊情况：由于坏字符在模式串中的最右位置可能在失配位置的右边，为了避免负数和 0 的情况，计算得到的偏移量要与 1 相比取较大的值.

3. 为什么一定找“最后一次出现位置”

如果模式串中某字符出现多次，选择最右边的那个位置来对齐，是为了获得尽可能大的安全位移. 选得更靠左，虽然也可能对齐，但会让移动步长变小，增加不必要的比较. 坏字符规则的算法思想是：**利用局部失配信息，尽量跳过不可能成功的状态.**

4. 预处理思想

为了快速知道“某个字符在模式串中最后一次出现在哪里”，需要先对模式串做一个预处理，建立“最后出现位置表”. 这个表也叫坏字符表。这个表的意义是：匹配过程中一旦遇到坏字符，就能立刻查表，不必重新扫描整个模式串.

例如：文本串 `T = "ABCACABDC"` ，模式串 `P = "abdc"` .

预处理：构造坏字符表 `last[i]` .

字符	a	b	d	c	其他
最后位置	0	1	2	3	-1

5. 伪代码实现

```
BM_BadChar(T, P)
输入：主串 T，模式串 P
输出：P 在 T 中第一次出现的位置；若不存在则返回 -1

n ← T.length
m ← P.length
若 m = 0，则返回 0
建立坏字符表 last
对所有字符 ch，令 last[ch] ← -1
for i ← 0 to m-1:
    last[P[i]] ← i

 s ← 0
 if s ≤ n - m:
   j ← m - 1
    if j ≥ 0 && P[j] = T[s+j]:
            j ← j - 1
    if j < 0:
        return s
    bad ← T[s+j]
    shift ← j - last[bad]
    if shift < 1:
        shift ← 1
    s ← s + shift

return -1
```

6. 复杂度分析

坏字符表的建立只需扫描一遍模式串，因此预处理时间通常是 $O(m)$.

搜索阶段最好情况的时间复杂度是 $O(n/m)$ ，最坏情况下为 $O(mn)$.

如果文本和模式串高度重复，或者模式串中大量字符重复，完整的 BM 算法还需要引入好后缀规则.